



A CPU-GPU Parallel Ant Colony Optimization Solver for the Vehicle Routing Problem

Antón Rey^(✉), Manuel Prieto, J. I. Gómez, Christian Tenllado,
and J. Ignacio Hidalgo

Departamento de Arquitectura de Computadores y Automática,
Facultad de Informática, Universidad Complutense de Madrid,
Prof. J. G. Santesmases, 9, 28040 Madrid, Spain
{anrey,mpmatias,jigomez,tenllado,hidalgo}@ucm.es

Abstract. This paper exposes a new hybrid approach based on Ant Colony Optimization heuristics, Route First-Cluster Second methods and Local search procedures, combined to generate high quality solutions for the Vehicle Routing Problem. This method uses the parallel computing power of modern general purpose GPUs and multicore CPUs, outperforming current ACO-based VRP solvers and showing to be a competitive approach compared to other high performing metaheuristic solvers.

Keywords: ACO · VRP · Parallel · Metaheuristic · GPU

1 Introduction

The basic definition of the Vehicle Routing Problem (VRP) asks for finding the optimal set of routes to be traversed by a fleet of vehicles in order to serve a set of customers. The VRP is one of the most studied and important combinatorial optimization problem even in its basic formulation. Its relevance relies on its applicability in logistic problems, since this situation arises naturally in most manufacturing or delivery companies. Multiple VRP variants are defined to model additional sets of constraints such as customer time windows or different capacity limitations for each vehicle. In this work we target the *Capacitated* VRP, in which all vehicles have the same capacity, although our approach can be generalized to other variants. In this case, any VRP feasible solution satisfies tree basic constraints: (1) all the customer nodes must be visited exactly once, (2) the aggregated demand of the customers visited by a vehicle must not exceed its capacity, and (3) every vehicle must start and end at a specific node, usually called the depot node.

Dantzig and Ramser introduced the problem and proposed the first algorithm to solve it in 1959. Since then, hundreds of exact and approximate algorithmic

approaches were developed. However, due to its NP-hard nature, the most efficient exact approaches hardly solves an instance with few hundred customers in an acceptable amount of time. Nevertheless, approximate approaches based on modern metaheuristics currently belong to the best performing solving methods, both in terms of quality of the solutions and computation time. The work of Blum and Roli [1] exposes a review of the role that metaheuristic approaches play in general combinatorial optimization problems. In particular, they highlight the importance of developing methods able to maintain a proper balance between the intensification and diversification components during the search: in order to find high quality solutions, a metaheuristic must explore efficiently the solution space via diversification mechanisms, while focusing at the same time on those promising regions through intensification techniques. Specifically, nature-inspired metaheuristics such as genetic-, evolutionary- and swarm-based algorithms among others, have proven to be excellent approaches in many combinatorial optimization problems. Hybridization between different approaches and integration of some metaheuristics with other traditional methods also proved to be useful in certain scenarios.

However, it is noteworthy that both approximate and exact methods demands a significant computational effort, since a typically vast solution space has to be efficiently explored. For this reason, the aid of parallel computing resources plays an important role in order to find high quality solutions in a short time.

In this context, we review some of the best performing VRP solvers. The work of [2] combines a Genetic algorithm approach with neighborhood-based metaheuristics reinforced with several specific diversification techniques. [3] exposes an intrinsically parallel algorithm based on the execution of multiple tabu searches in parallel that cooperate and maintain a dynamic balance in diversification and intensification. In the thesis of Groer [4] it is exposed another hybrid parallel approach and given an extensive overview of both parallel and serial methods. The works in [5] and [6] hybridize a GRASP approach with both iterative and evolutionary-based local search methods. Subramanian exposes in [7] an extensive study about both exact and heuristic methods, while also presenting a highly competitive approach based on the Iterated Local Search metaheuristic that uses a Variable Neighbourhood Descent, capable of handling several VRP variants.

This paper exposes a hybrid metaheuristic based on the Max Min Ant System (MMAS) [8] method derived from Ant Colony Optimization (ACO) approaches and some deterministic methods designed to build and optimize VRP solutions constructed from Traveling Salesman Problem (TSP) routes related to the VRP problem. In this approach, we show that the MMAS, originally designed for the TSP, is capable of generating through the usual operations of ACO algorithms the source of stochasticity needed to explore efficiently –but indirectly– the search space of the Vehicle Routing Problem.

From the implementation perspective, our approach exploits different levels of parallelism. A set of TSP routes is constructed in parallel by an ACO algorithm

implemented in GPU, in which every ant builds a single TSP route exploiting a lower level of parallelism. Next, a CPU thread builds a complete VRP solution from each TSP route, using a Route First-Cluster Second (RFCS) algorithm [9]. Next, each CPU thread improves its VRP solution by applying Local search (LS) procedures based on inter- and intra-route optimizations.

Summarizing our contributions, we present a new parallelization strategy for the Ant Colony Optimization to the Traveling Salesman Problem, a new metaheuristic for the Vehicle Routing Problem, and we expose the parallelization of the whole algorithm through a heterogeneous CPU-GPU implementation.

2 Method

We propose a new approach for the Capacitated Vehicle Routing Problem based on Ant Colony Optimization [8] framework integrated with a Route First-Cluster Second method [9] and followed by specific inter-route based on λ -interchanges [10] and intra-route k -opt intensification mechanisms.

Given a set of locations, the TSP asks for finding the least cost route that visits each location exactly once. Any route that satisfies this criteria but it is not necessarily optimal is referred as a *TSP route* or *TSP solution*. Similarly, a feasible set of vehicle routes that satisfies the constraints of a VRP instance will be referred as a *VRP solution*. Note that any TSP route can be regarded as a VRP solution in which just a single vehicle with enough capacity to serve all customers is available. Similarly, taking into account the corresponding VRP constraints, a given TSP route can be partitioned into a set of subroutes to form a feasible VRP solution. Specifically, the VRP solution is built from a least cost path calculated on a transformed graph that depends on the VRP constraints, as it is explained in Sect. 2.2. On top of these ideas, we designed an iterative process meant to find a TSP tour whose RFCS split and consequent Local search optimizations yields the optimal VRP solution. Similar methods were already implemented by [5,6]. They both used a GRASP instead of our ACO method to generate the TSP routes to split and applied different local search techniques in the post-optimization stage. They showed that it was a promising alternative among the best performing VRP and some of its variants. In our case, the process of generating TSP tours is made by an Ant Colony Optimization algorithm. First, the ACO algorithm generates a set of TSP routes that are split independently in a set of subroutes to form an intermediate set of VRP solutions. Once all VRP solutions are improved, the arcs that belong to the best found solution or the best solution at current iteration will be more desirable for the ants to be chosen at the next step. Next, we expose the main stages of our method.

2.1 Ant Colony Optimization Overview

The first stage of the iterative process implements Ant Colony Optimization techniques and it is the only source of stochasticity of the whole method.

The ACO algorithms construct routes in a graph, imitating the half-erratic/half-guided movement of natural ants. The basic stages of a simple ACO algorithm are exposed in Algorithm 1.

Algorithm 1. General ACO algorithm for the Traveling Salesman Problem

Initialize pheromone trails.
while Stop condition is not met **do**
 Every ant builds a TSP route favoring arcs with high amount of pheromone.
 Compute the costs of each TSP route.
 Add pheromone on those arcs belonging to the best route.
 Evaporate pheromone on all arcs.
end while

Each route is constructed starting from a node and adding neighboring nodes sequentially. Being in a node i , the probability of choosing a certain node j in i 's neighborhood N_i is given by:

$$p_{ij} = \frac{(\tau_{ij})^\alpha (\eta_{ij})^\beta}{\sum_{l \in N_i} (\tau_{il})^\alpha (\eta_{il})^\beta} \text{ if } j \in N_i,$$

where η_{ij} represents the *heuristic information* (usually set as the inverse of the distance from the node i to the node j), and τ_{ij} represents the *amount of pheromone* on the arc ij . The greater the pheromone and heuristic information values associated to an ij arc, the more likely is that node j will be chosen after node i is visited. α and β are parameters that quantify the relative importance of the pheromone information versus the heuristic information.

The following stage of an ACO algorithm comprises the pheromone update of the arcs. First, the amount of pheromone evaporation of a given arc that links the nodes i y j is set by the rule

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij}, \forall (i, j),$$

which depends on an evaporation rate parameter ρ ($0 < \rho < 1$). Moreover, given a set of arcs $(i, j \in K)$ belonging to a certain route K with positive cost C , the amount of pheromone added on them is defined by:

$$\tau_{ij} \leftarrow \tau_{ij} + 1/C, \forall (i, j) \in K.$$

Among the ACO methods for the TSP solving, the one which exhibits the best balance between exploration and exploitation is the Max-Min Ant System (MMAS) [8], which introduces four main modifications respect to the most basic Ant System (AS). First, it exploits the best routes found: only those arcs belonging to the best route found either at (A) current iteration or (B) the best found route, get their pheromone increased, thus increasing the probability of being chosen in next iterations. The first option (A) makes the ants explore the solution

space (exploration phase), while the second option forces all ants to concentrate the exploration in a neighborhood of a given solution (exploitation phase). It is suggested [8] that the exploration phase has to be done at the beginning of the convergence process while the exploitation phase has to be activated once a given number of iterations is reached. This transition is suggested to be smooth, for instance by increasing the frequency in which the pheromone of the best-so-far solution is added.

Stagnation Control. Nevertheless, with these simple mechanisms the convergence may get interrupted due to solution stagnation, in which all ants will eventually follow the same route, due to an excessive pheromone growing on those arcs belonging to a suboptimal route. To counteract this effect, a second modification keeps the pheromone amount within a range $[\tau_{min}, \tau_{max}]$. At the beginning of the algorithm all arcs are initialized to the upper bound, so that the algorithm can explore the search space in a diversified manner. Finally, the pheromone trails are reinitialized again to the upper bound if any stagnation condition is met or when a certain number of iterations is reached.

We want to highlight that we do not need to modify the normal MMAS design in the TSP route construction stage. We can explore the VRP solution space just using the mentioned ACO TSP parameters in the same way. We only added one ad-hoc condition to force a pheromone reset when stagnation is detected, happening when the VRP solutions are not sufficiently diversified. This situation arises either when all arcs of the graph have the minimum amount of pheromone –except the ones that belong to the best known solution–, or when a certain number of iterations is reached without improvement. For these cases, pheromone of all arcs with τ_{min} pheromone is raised to $95\% \tau_{max}$ so these arcs are more likely to be chosen again in the next iteration. Doing this we make the iterative process to *jump* outside a local suboptimal solution while still keeping memory of the best found solutions –as their arcs pheromone is kept at τ_{max} –.

2.2 Splitting Procedure via Route First-Cluster Second Method

As mentioned, a TSP route can be divided with the RFCS [9] splitting procedure in an optimal way, so the attractiveness of the method lies in the exploration of a *simpler* TSP solution space.

The method works as follows. Assigning a particular direction to the TSP route, letting the index $i + a$ be the a^{th} node visited after the node i (assigning 0 to the depot node), and being d_{ij} the components of the distance matrix, another matrix c_{ij} is built following the rules:

- c_{ij} = Distance traveled by a vehicle following the route $(i + 1, i + 2, \dots, j)$ if the route $(0, i + 1, i + 2, \dots, j, 0)$ is feasible ($i < j$), or
- $c_{ij} = \infty$ If the same route is infeasible.

Namely, if the aggregated customer demands in route $(0, i + 1, i + 2, \dots, j, 0)$ can be satisfied by a single vehicle, then

$$c_{ij} = d_{0(i+1)} + \sum_{k=i+1}^{j-1} d_{k(k+1)}.$$

Considering first an homogeneous fleet where all vehicles have the same capacity, if we compute the least cost path from node 0 to node N in the graph defined by the arc costs c_{ij} , we would have computed an optimal split of the initial directed tour. If the least cost path calculated has m arcs, this path would correspond to a VRP solution with m vehicles. Contrary, if no route from 0 to N exists, then the problem is infeasible.

Note that setting a maximum length per route L_{max} , and thus solving a Distance Constrained VRP (DCVRP), would simply require to add the following condition:

- $c_{ij} = \infty$ if the length of the route $(0, i + 1, i + 2, \dots, j, 0)$ is greater than L_{max} .

Note that it can also be easily generalized to the heterogeneous fleet case, which would imply the resolution of a Resource Constrained Shortest Path Problem. Other VRP constraints can be modeled similarly, which illustrate the versatility of this method for solving other VRP variants.

2.3 Local Search Optimizations

After splitting a TSP route, every VRP solution is improved independently via inter-route optimizations based on λ -interchanges [10]. These interchanges apply *shift* and *swap* operations between groups of nodes assigned to a pair of vehicles.

Algorithm 2. Pseudocode for a given λ -interchange operator.

```

for every VRP solution do
  for every subroute  $r_A$  do
    for every subroute  $r_B$  do
      if  $r_A \neq r_B$  then
        Loop over  $r_A$  and  $r_B$  nodes and evaluate all possible shift/swap movements.
        Make the movement that leads to the best improvement.
      end if
    end for
  end for
end for

```

Shift movements comprise movements of blocks of nodes: a *shift*($\lambda, 0$) subroutine moves λ adjacent nodes from route A to route B . Swap movements involve swapping of nodes: a *swap*(λ, μ) movement interchanges λ adjacent nodes of route A with μ adjacent nodes of route B . The larger the arguments of both

shift and swap procedures, the better the potential improvements, while also implying more computational load. The implementation of the first two simplest shifts ($\lambda = 1, 2$) and the three simplest swaps ($(\lambda, \mu) = (1, 1), (2, 1), (2, 2)$) was enough to achieve good performance while keeping the computational load not too high. This stage not only helps to achieve best cost solutions, but it also helps in minimizing the number of vehicles. After these inter-route improvements, an intra-route improvement stage is made by applying the well known 2-opt or 3-opt procedures to every vehicle route, which finds the best interchanges between two or three arcs, respectively. k -opt operators with $k > 3$ are rarely considered because their typical improvements do not worth their increase in computational complexity. Instead, 2-opt and 3-opt operators implementing the usual speed-up techniques provided a good balance between solution improvement and processing time.

2.4 LKH Initialization

Before starting the iterative process, a pheromone initialization is done from an optimal or near optimal solution of the TSP associated to the VRP graph. The TSP solver by Helsgaun [11] (*LKH*) provides optimal or high quality TSP solutions for problems up to several thousands of nodes in a very short time. By setting extra pheromone on those arcs belonging to this TSP route found by the LKH solver, we get good VRP solutions to start with, which speeds up the convergence process of the algorithm.

Finally, it can be noticed that the whole iterative process involves a considerable computational load specially if many ants are run. As a matter of fact, this method is population-based: its good convergence behavior depends on handling a population with several number of members/ant/VRP solutions. From the computational perspective, although the RFCS stage is relatively lightweight, the following stages running local search optimizations imply remarkable computing demands. For this reason, the use of the parallel processing is highly desirable. In Sect. 3 it is shown that the TSP route construction motivates the use of the general purpose GPUs while the VRP-specific stages can be targeted via CPU threads.

3 Parallel Implementation

In this section we describe the parallel implementation of the two main stages performed at each iteration. The source code under MIT license can be found in <https://github.com/aarv87/aco-rfcs-vrp>.

3.1 ACO on GPU for TSP Routes

Fortunately, the Max-Min Ant System (MMAS) is the ACO variant with best convergence to optimum for the TSP and also the best candidate for implementation on GPU. A number of m blocks of GPU threads is assigned to perform

the computation associated to m ants. Specifically, the ant decision process can be parallelized in a way such that every GPU thread computes the probability p_{ij} of visiting every candidate node j when the ant is in node i : a $O(nn)$ task is parallelized being nn the number of candidate nodes or nearest neighbors. Once all the probabilities of the nn nodes are calculated, the ant may choose one candidate using the standard roulette algorithm; however this is not well suited for a parallel GPU implementation. Instead, the GPU-specific *All-In-Roulette* approach exposed in [12] was implemented. Moreover, in the MMAS approach only those arcs belonging to a single route are the ones where the pheromone is increased. Thus the pheromone addition on $n - 1$ independent arcs of the n -nodes route can be distributed over $n - 1$ parallel GPU threads straightforwardly.

GPU data is arranged as matrices whose sizes depend on the number of nodes n of the problem instance, the number of nearest neighbors considered nn and the number of ants m .

- Distance matrix: $[n \times n]$
`distance[i][j]` represents the distance or cost from node i to node j .
- Pheromone matrix: $[n \times nn]$
`pheromone[i][j]` stores the amount of pheromone of the arc that connects node i with node j .
- Probability matrix: $[n \times nn]$
`prob[i][j]` represents the probability of choosing the node j being at node i . It is computed from the pheromone and distance matrices, and α, β parameters.
- Taboo matrix: $[m \times n]$
`taboo[i][j]` equals 0 if the ant i has visited node j and 1 if not.
- Solutions matrix: $[m \times (n + 1)]$
`traveled[i][j]` stores for each ant i the index of the node visited in the j^{th} place of its route. Since the routes are closed, `traveled[i][1] = traveled[i][n+1]  $\forall i$` .
- Random matrix: $[m \times nn]$
`dice[i][j]` stores random numbers uniformly distributed in $(0, 1]$.

The main idea in the GPU parallelization relies on mapping every matrix row into each block i of threads, in such a way that its threads get mapped in an aligned way to its columns, thus facilitating high throughput transfers from/to GPU memory. If a certain matrix has C columns, in general every thread j of the block i reads or writes every element $j + b \times nTpB$, being b a loop variable $\in [0, C/nTpB]$.

In the route construction stage, every block of threads is responsible for constructing every solution. Supposing the ant i is located at node `traveled[i][k]`, the calculation of each total probability element `TP[i][j]` is made in parallel by:

$$TP[i][j] \leftarrow taboo[i][j] * dice[i][j] * prob[traveled[i][k]][j],$$

where each thread j of the block i performs the computation for the candidate j of the current node `traveled[i][k]`. Once all the total probabilities of all

candidate nodes are computed, **traveled** matrix is updated with the next node j^* to be visited, by doing a parallel reduction over the corresponding i -row elements of the TP matrix to get the maximum element:

traveled[i][$k+1$] $\leftarrow j^*$.

Analogously, once all routes are constructed, each block with index i is responsible for the total distance computation of its route i making the corresponding parallel reduction in on-chip shared memory. In this case, aligned thread-to-element initial reads of **distance** matrix elements will not be satisfied in general. However, some locality could be exploited if the proximity of nodes in the graph is correlated with the proximity in their index number.

For the update of **pheromone** and **prob** matrices, every thread j is straightforwardly mapped to every arc $(j, j+1)$ without the need of any synchronization.

3.2 RFCS + Local Search on CPU Threads for VRP Solutions

For each TSP route, a CPU thread performs a RFCS split and consequent Local search optimizations on it. CPU threads are synchronized at the end of this stage before selecting the best VRP solution. Note that VRP solutions might be different in size, so the duration in their Local search stage can be very different among each other, making some threads to wait for others to finish.

4 Experimental Results

In this section we first expose metrics for scalability to evaluate the parallel performance in both CPU and GPU stages, running sequential executions as references, and show the impact of these stages in the overall time to solution. Secondly, we run our solver on a set of well-known VRP instances to compare the solving performance of our approach against ACO-based solvers (Tables 1 and 2) and against some of the best performing solvers (Tables 3 and 4) –values in bold correspond to optimal solutions–.

4.1 CPU and GPU Scalability and Relative Computation Times

First we expose in Fig. 1 the scalability of the RFCS + LS stage, where VRP solutions are constructed and refined via RFCS and local search methods by each thread (see Sect. 3.2). For each problem we run 5 executions for different number of threads: 4, 8 and 16 threads executions were run on a CPU with 2×4 cores, mapping explicitly each thread to a single processor for the 4 and 8 cases and enabling Simultaneous Multithreading (SMT) for the 16 threads case. This was done analogously on the 2×6 CPU machine, enabling the SMT thread scheduling for the 24 threads case. We calculated the speed-up dividing the sequential time by the parallel time measured when running 100 iterations.

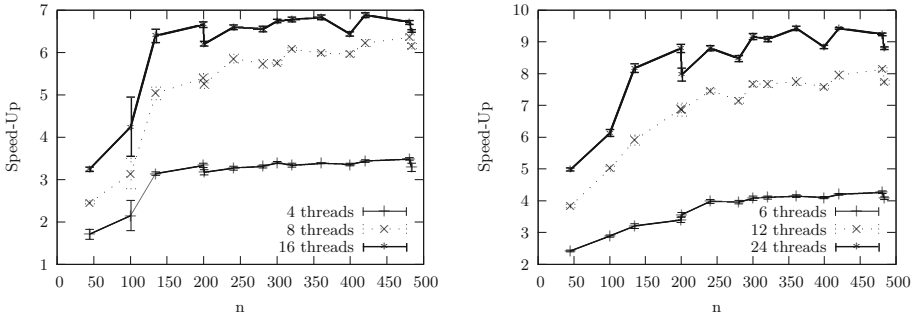


Fig. 1. CPU Speed-Up results for (left) 4, 8 and 16 threads/VRP solutions in 2×4 CPU Intel Xeon X5570 *Nehalem* @ 2.93Ghz and (right) for 6, 12 and 24 threads/VRP solutions in 2×6 CPU Intel Xeon E5645 *Westmere* @ 2.40GH.

It is worth noting that every thread constructs and optimizes independently its own VRP solution from a diversified set of TSP routes, so the VRP solutions may be very different among them. This fact implies that the optimizations done in the subsequent λ -interchanges and 3-opt operations may take very different times depending on the VRP solution, which causes load imbalance among threads, thus making some of them to wait for others to finish.

Note that the higher the number of nodes, the greater is the speed-up obtained until saturation at around 200 nodes. A reasonable good ratio between the number of threads and the speed-up is seen when a single thread is assigned to each physical core –i.e. SMT disabled–, noticing that speed-up practically doubles from 4 to 8 threads (Fig. 1 left) and from 6 to 12 threads (Fig. 1 right).

Secondly, we show how the speed-up of the ACO stage increases with the number of nodes of the problem. Note that in this analysis the number of vehicles does not play any role, since the ACO stage constructs TSP routes considering only the location of the nodes in the graph. Similarly, our GPU parallel implementation of the ACO-MMAS method was compared with the sequential version of Dorigo and Stutzle ([8]). Both computation times were measured using the same set of parameters that are relevant on the computation times, taking into account only the main five ACO stages: (1) initialization, (2) solution construction, (3) cost calculation, (4) best solution checking and (5) pheromone updating. In the parallel executions, the number of ants equals the number of blocks of threads, and the number of candidates during the solution construction defines the number of threads per block, as explained in Sect. 3.1.

A linear behavior of the speed-up factor with the size of the problem can be observed in Fig. 2. In fact, a saturation plateau in the speed-up was not reached for the instance sizes tested, suggesting that the GPU resources are not fully utilized for this set of instances.

Third, we compare how the relative computation times of both stages ACO-GPU and RFCS-CPU depend on the size of the problem, both in the sequential and parallel cases. In Fig. 3 it can be seen that VRP stage dominates the

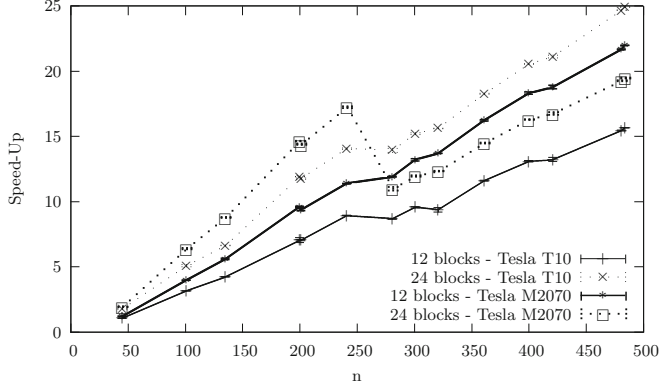


Fig. 2. Speed-Up results for the ACO stage in which TSP routes are constructed, comparing sequential implementation and single GPU implementation.

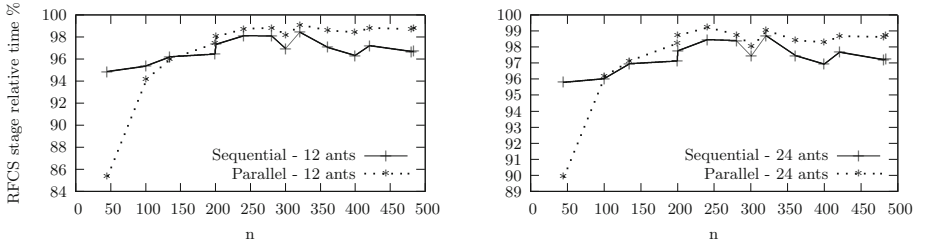


Fig. 3. Relative times of the sequential and parallel RFCS + LS stage, generating 12 and 24 VRP solutions per iteration.

computation –specifically, the Local search computation dominates the RFCS split–. In fact, the relative computation time of the CPU stage grows rapidly with the size of the problem in the parallel implementation, while it remains approximately stable around 96% of the total time in the sequential implementation. It should be noticed that the LKH initialization stage has not been taken into account because of its negligible impact in the overall time to solution for the instances studied.

4.2 Convergence to Optimum

In this section we study the convergence behavior of our algorithm in terms of the average quality for Christofides [6] and Golden [13] instances. Measurements of convergence time are also exposed but they should not be used for comparing the different solvers, since each one is tested on different hardware. First, we compare our method with the best ACO-based VRP solvers in Tables 1 and 2, showing that it provides the best average convergence among all of them. In Tables 3 and 4 our method is compared with some of the best performing VRP solvers,

Table 1. Comparison of ACO-RFCS with best ACO-based VRP solvers in Christofides [6] instances.

	BKS	[14]	[15]	[16]	[17]	ACO-RFCS
C1	524.61	528.90	524.61	526.65	—	524.61
C2	835.26	—	840.61	850.27	—	836.87
C3	826.14	865.65	828.21	849.32	—	828.55
C4	1028.42	1123.28	1037.57	1051.90	1049.80	1030.70
C5	1291.29	—	1306.91	1351.54	1334.73	1306.09
C6	555.43	—	555.43	562.53	—	555.43
C7	909.68	—	917.50	923.07	—	910.30
C8	865.94	—	865.94	877.59	—	866.74
C9	1162.55	—	1173.94	1182.97	—	1166.46
C10	1395.85	—	1415.53	1453.92	—	1405.54
C11	1042.11	—	1043.46	1044.64	—	1042.11
C12	819.56	—	819.56	843.93	—	819.56
C13	1541.14	—	1546.84	1553.75	—	1544.43
C14	866.36	—	866.36	867.62	—	866.36
	Avg. gap (%)	4.94	0.48	1.87	2.72	0.23

Table 2. Comparison of ACO-RFCS with best ACO-based VRP solvers in Golden [13] instances.

	BKS	[15]	[17]	ACO-RFCS
D241-10k	5623.47	5690.67	-	5718.68
D321-10k	8404.61	8476.64	-	8628.88
D401-10k	11036.22	11117.89	-	11188.68
D481-12k	13590.00	13755.32	-	13662.62
D201-05k	6460.98	6460.98	-	6460.98
D281-08k	8400.33	8412.90	-	8412.90
D361-09k	10102.70	10209.86	-	10229.94
D441-11k	11635.30	11867.81	-	11910.28
E256-14k	579.71	590.14	-	586.64
E324-16k	735.66	752.76	-	749.28
E400-18k	912.03	942.01	-	928.34
E484-19k	1101.50	1144.33	-	1126.11
E253-27k	857.19	873.84	-	862.31
E321-30k	1080.55	1097.86	-	1085.02
E397-34k	1337.87	1367.29	-	1354.17
E481-38k	1611.56	1654.55	-	1640.69
E241-22k	707.76	711.11	-	709.83
E301-28k	995.13	1010.64	1034.02	1011.13
E361-33k	1365.60	1379.38	1414.3	1372.73
E421-41k	1817.89	1840.46	1890.96	1840.93
	Avg. gap (%)	1.56	3.83	1.24

Table 3. Comparison of ACO-RFCS with best VRP metaheuristics in Christofides [6] instances. Average time in seconds.

	[5]		[4]	[2]		[7]		ACO-RFCS			
BKS	Av.cost	Av.t	Av. cost	Av. cost	Av.t	Av. cost	Av.t	Best cost	Av.cost	Av.t	
C1	524.61	524.61	0.06	524.61	524.61	177.60	524.61	2.11	524.61	524.61	1.65
C2	835.26	835.26	15.01	835.26	835.26	298.80	835.73	5.49	835.26	836.87	118.80
C3	826.14	826.14	3.19	826.14	826.14	379.80	826.33	12.87	826.14	828.55	290.69
C4	1028.42	1029.48	44.84	1028.42	1028.42	639.00	1032.16	39.75	1029.56	1030.70	414.71
C5	1291.29	1294.09	11.85	1291.45	1291.74	1171.20	1305.43	92.25	1297.95	1306.09	986.51
C6	555.43	555.43	0.14	555.43	555.43	196.80	556.84	1.43	555.43	555.43	1.90
C7	909.68	909.68	5.18	909.68	909.68	353.40	910.70	6.24	909.68	910.30	334.75
C8	865.94	865.94	0.32	865.94	865.94	400.80	866.07	10.21	865.94	866.74	22.01
C9	1162.55	1162.55	6.58	1162.99	1162.55	720.00	1167.78	31.65	1162.55	1166.46	329.47
C10	1395.85	1401.46	25.57	1398.52	1397.70	1985.40	1411.50	69.40	1401.55	1405.54	1000.91
C11	1042.11	1042.11	0.64	1042.11	1042.11	399.00	1042.11	23.58	1042.11	1042.11	11.59
C12	819.56	819.56	0.17	819.56	819.56	303.60	819.56	7.69	819.56	819.56	2.86
C13	1541.14	1545.43	9.25	1542.86	1542.86	759.00	1544.57	18.61	1542.86	1544.43	1069.71
C14	866.37	866.37	0.27	866.37	866.37	409.80	866.37	8.54	866.37	866.37	3.17
	Av.gap(%)0.07			0.03	0.02		0.27		0.08	0.23	

Table 4. Comparison of ACO-RFCS with VRP metaheuristics in Golden [13] instances. Average time in seconds.

BKS	[5]		[3]		[2]		[7]		ACO-RFCS		
	Av.cost	Av.t	Av.cost	Av.t	Av.cost	Av.t	Av.cost	Av.time	Best cost	Av.cost	Av.t
5623.47	5644.52	3.97	5623.65	22.05	5625.10	66.90	5711.19	2.53	5646.58	5718.68	6.68
8404.61	8447.92	1.55	8434.78	34.22	8419.25	103.91	8480.62	7.37	8623.48	8628.88	21.64
11036.22	11036.22	6.42	11036.22	44.64	11036.22	116.43	11076.20	15.90	11036.22	11188.68	4.06
13590.00	13624.52	5.83	13620.30	60.87	13624.52	175.22	13665.85	28.38	13624.50	13662.62	10.52
6460.98	6460.98	1.05	6460.98	15.83	6460.98	13.25	6460.98	1.88	6460.98	6460.98	0.16
8400.33	8412.90	1.41	8404.06	26.76	8412.90	41.95	8412.90	5.55	8412.90	8412.90	0.77
10102.70	10195.59	7.72	10134.93	39.01	10134.90	108.49	10228.80	11.35	10195.60	10229.94	13.90
11635.30	11643.90	4.97	11635.34	54.61	11635.30	152.42	11842.80	21.54	11828.80	11910.28	23.97
579.71	586.23	1.31	580.04	19.43	581.08	51.69	589.54	4.34	584.93	586.64	39.51
735.66	744.36	7.30	737.16	28.82	738.92	104.02	749.16	9.98	744.85	749.28	41.91
912.03	922.40	4.55	912.72	41.33	914.37	131.23	928.23	20.51	925.01	928.34	47.90
1101.50	1116.12	29.11	1103.20	58.29	1105.97	196.52	1121.24	49.50	1124.37	1126.11	43.80
857.19	862.32	2.14	858.57	18.06	859.08	25.56	865.64	2.85	860.16	862.31	37.87
1080.55	1089.35	9.12	1080.55	25.08	1081.99	43.35	1092.07	5.74	1081.31	1085.02	20.43
1337.87	1352.39	10.22	1340.13	36.33	1341.95	109.14	1354.24	13.07	1349.84	1354.17	50.82
1611.56	1634.27	21.05	1614.73	48.14	1616.92	174.70	1635.62	20.79	1634.74	1640.69	52.41
707.76	708.85	1.03	707.80	16.39	707.84	28.98	709.21	2.95	708.99	709.83	29.34
995.13	1002.15	2.93	998.90	25.01	996.95	40.83	1003.13	8.16	1007.46	1011.13	39.79
1365.60	1371.67	17.98	1366.12	32.60	1366.39	81.49	1376.01	11.86	1368.21	1372.73	39.88
1817.89	1830.98	5.77	1819.76	41.93	1819.75	88.31	1836.06	21.91	1834.91	1840.93	53.10
Av. gap (%)	0.65		0.12		0.18		1.06		0.85	1.24	

exposing average behaviors for all approaches and also best-case behavior for our ACO-RFCS solver. The experimental results are not definitive, but we can affirm that the proposed method gives comparable results in terms of average quality and excellent results in the best cases.

5 Conclusions

In this paper we have presented a hybrid parallel implementation of a VRP iterative solver composed of two stages. The first stage applies Ant Colony Optimization MMAS method to generate TSP routes in the GPU. The second stage is run by CPU threads and applies the Route First-Cluster Second method to split each TSP route into a VRP solution, which is subsequently optimized via inter- and intra-route Local search procedures. The convergence is guided by making the arcs belonging to the best found VRP solutions more likely to be chosen in the TSP route generation of the next iteration. We have provided metrics for parallel scalability and shown that on average our solver is able to find high quality solutions on two sets of VRP instances. As future work, convergence time is expected to be improved by (1) exploiting concurrent execution capabilities in some GPU devices and (2) managing a concurrent pool of VRP solutions. With these improvements CPU-GPU synchronization is relaxed and idle times in CPU threads are expected to be greatly reduced.

Acknowledgements. This work has been supported by the EU (FEDER) and the Spanish MINECO, under grants TIN2014-54806-R, TIN2015-65277-R and BES-2016-076806.

References

1. Blum, C., Roli, A.: Metaheuristics in combinatorial optimization: overview and conceptual comparison. *ACM Comput. Surv. (CSUR)* **35**(3), 268–308 (2003)
2. Vidal, T., Crainic, T., Gendreau, M., Lahrichi, N., Rei, W.: A hybrid genetic algorithm for multidepot and periodic vehicle routing problems. *Oper. Res.* **60**(3), 611–624 (2012)
3. Jin, J., Crainic, T., Lokketangen, A.: A Cooperative Parallel Metaheuristic for the Capacitated Vehicle Routing Problem (2012)
4. Groer, C.: Parallel and serial algorithms for vehicle routing problems. In: *ProQuest* (2008)
5. Prins, C.: A grasp $\times$ evolutionary local search hybrid for the vehicle routing problem. In: Pereira, F.B., Tavares, J. (eds.) *Bio-inspired Algorithms for the Vehicle Routing Problem*, *Studies in Computational Intelligence*, vol. 161, pp. 35–53. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-540-85152-3_2
6. Christophe, D., Philippe, L., Prodron, C., et al.: A GRASP $\times$ ELS with Depth First Search Split Procedure for the HVRP (2012)
7. Subramanian, A.: Heuristic, Exact and Hybrid Approaches for Vehicle Routing Problems. Ph.D. thesis, Universidade Federal Fluminense, Niterói, Brazil (2012)
8. Dorigo, M., Birattari, M., Stutzle, T.: Ant colony optimization. *IEEE Comput. Intell. Mag.* **1**(4), 28–39 (2006)

9. Beasley, J.: Route first–cluster second methods for vehicle routing. *Omega* **11**(4), 403–408 (1983)
10. Osman, I.: Metastrategy simulated annealing and tabu search algorithms for the vehicle routing problem. *Ann. Oper. Res.* **41**(4), 421–451 (1993)
11. Helsgaun, K.: An effective implementation of the lin-kernighan traveling salesman heuristic. *Eur. J. Oper. Res.* **126**(1), 106–130 (2000)
12. Guohua, F.: Parallel ant colony optimization algorithm with GPU-acceleration based on all-in-roulette selection. *Comput. Digital Eng.* **5**, 007 (2011)
13. Golden, Bruce L., Wasil, Edward A., Kelly, James P., Chao, I-Ming: The impact of metaheuristics on solving the vehicle routing problem: algorithms, problem sets, and computational results. In: Crainic, Teodor Gabriel, Laporte, Gilbert (eds.) *Fleet Management and Logistics. CRT*, pp. 33–56. Springer, Boston, MA (1998). https://doi.org/10.1007/978-1-4615-5755-5_2
14. Bell, J., McMullen, P.: Ant colony optimization techniques for the vehicle routing problem. *Adv. Eng. Inform.* **18**(1), 41–48 (2004)
15. Reimann, M., Doerner, K., Hartl, R.: D-ants: savings based ants divide and conquer the vehicle routing problem. *Comput. Oper. Res.* **31**(4), 563–591 (2004)
16. Chen, C., Ting, C.: An improved ant colony system algorithm for the vehicle routing problem. *J. Chin. Inst. Ind. Eng.* **23**(2), 115–126 (2006)
17. Lucka, M., Piecka, S.: Ant colony optimizer with application to the vehicle routing problem. *J. Appl. Math.* **4** (2011)